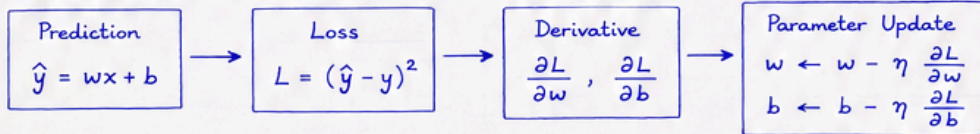


Loss Functions and Gradient Descent

Page 1



Our goal: minimize the loss by updating parameters using gradients. 😊

① The simplest possible model

One input x , one weight w , one bias b .

$$\hat{y} = wx + b$$

$\hat{y}$: prediction
 y : true value
 w, b : parameters

Example: $x = 2, y = 5, w = 1, b = 0$

$$\hat{y} = (1)(2) + 0 = 2 \quad (\text{wrong prediction})$$

② Loss function (Squared Error)

How wrong is the prediction?

$$L = (\hat{y} - y)^2$$

Here, $\hat{y} = 2, y = 5 \Rightarrow L = (2 - 5)^2 = 9$

Why square?

- Errors of opposite signs will not cancel. $(7-5)=2 \rightarrow 4$
- Larger errors are penalized more. $(3-5)=-2 \rightarrow 4$

③ Objective

For fixed data (x, y) , the loss is a function of (w, b) :

$$L = (wx + b - y)^2 \equiv L(w, b)$$

Training means: Find w and b that minimize $L(w, b)$.

④ Derivatives (Gradients)

We need $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$.

$$L = (wx + b - y)^2$$

• For w :

Let $u = wx + b - y \Rightarrow L = u^2$

$$\frac{\partial L}{\partial u} = 2u, \quad \frac{\partial u}{\partial w} = x$$

$$\therefore \frac{\partial L}{\partial w} = 2u \cdot x = 2(wx + b - y)x = 2(\hat{y} - y)x$$

• For b :

$$\frac{\partial \hat{y}}{\partial b} = 1, \quad \frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

$$\therefore \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

⑤ Gradient Descent Update Rule

$$\begin{aligned} w_{\text{new}} &= w_{\text{old}} - \eta \frac{\partial L}{\partial w} \\ b_{\text{new}} &= b_{\text{old}} - \eta \frac{\partial L}{\partial b} \end{aligned} \quad (\eta: \text{learning rate})$$

We subtract the gradient because $-\nabla L$ points in the direction of steepest decrease of the loss.

⑥ Numerical Example (one training point)

Take $x = 2, y = 5, w = 1, b = 0, \eta = 0.1$

Step 1: Forward pass

$$\hat{y} = wx + b = (1)(2) + 0 = 2$$

Step 2: Loss

$$L = (\hat{y} - y)^2 = (2 - 5)^2 = 9$$

Step 3: Gradients

$$\begin{aligned} \frac{\partial L}{\partial w} &= 2(\hat{y} - y)x = 2(2 - 5)(2) = -12 \\ \frac{\partial L}{\partial b} &= 2(\hat{y} - y) = 2(2 - 5) = -6 \end{aligned} \quad \left. \begin{array}{l} \text{(negative} \rightarrow \\ \text{increase } w, b \end{array} \right\}$$

Step 4: Update

$$\begin{aligned} w_{\text{new}} &= 1 - 0.1(-12) = 2.2 \\ b_{\text{new}} &= 0 - 0.1(-6) = 0.6 \end{aligned}$$

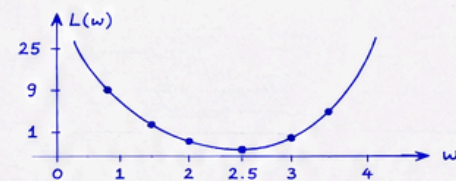
Step 5: New prediction and loss

$$\begin{aligned} \hat{y}_{\text{new}} &= (2.2)(2) + 0.6 = 5 \\ L_{\text{new}} &= (5 - 5)^2 = 0 \end{aligned}$$

Loss decreased from 9 $\rightarrow$ 0
Good! 😊

⑦ Loss landscape for one parameter ($b = 0$)

For $x = 2, y = 5, L(w) = (2w - 5)^2$



Minimum at $w = 2.5$
 $\rightarrow L = 0$ 😊

Gradient tells us the slope (direction).

Gradient descent moves toward the minimum.

⑧ Multiple Training Examples $\rightarrow$ Mean Squared Error (MSE)

Dataset: $(x_i, y_i), i = 1, \dots, N$

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad \text{where } \hat{y}_i = wx_i + b$$

Gradients:

$$\frac{\partial L}{\partial w} = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i)x_i, \quad \frac{\partial L}{\partial b} = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i)$$

⑨ Training Loop in NumPy (Linear Regression)

```
import numpy as np
# Training data
x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([3.0, 5.0, 7.0, 9.0]) # y = 2x + 1
# Initialize parameters
w = 0.0 # b = 0.0
# Hyperparameter
eta = 0.01 # learning rate
N = len(x)
for epoch in range(1000):
    # Forward pass
    y_hat = w * x + b
    error = y_hat - y
    loss = np.mean(error**2) # MSE
    # Gradients
    dL_dw = (2/N) * np.sum(error * x)
    dL_db = (2/N) * np.sum(error)
    # Update
    w = w - eta * dL_dw
    b = b - eta * dL_db
    # Print every 100 epochs
    if epoch % 100 == 0:
        print(f"epoch {epoch:4d} Loss={loss:.6f} w={w:.4f} b={b:.4f}")
print(f"Final w=", w, ", Final b=", b)
```

The parameters converge to $w \approx 2$ and $b \approx 1$.

This code is pure NumPy.
No ML library!
😊

⑩ Effect of Learning Rate η

η (learning rate)	Behavior	Sketch
Very small (e.g. 0.0001)	Very slow convergence	
Good choice (e.g. 0.01)	Smooth and fast convergence	
Too large (e.g. 1.0)	Overshoot, may diverge	

⑪ Key Ideas to Remember

- Loss tells us how bad the prediction is.
- Gradient tells us how to change parameters.
- Gradient descent updates parameters in the direction of steepest decrease of the loss.
- Training a neural network = an optimization problem.
- This simple idea scales to millions or billions of parameters using backpropagation (next lesson!).

⑫ Your Exercise (Do $i = 1$ to N , $b = 1, \eta = 0.05$)

- $\hat{y} = wx + b = ?$
- $L = (\hat{y} - y)^2 = ?$
- $\frac{\partial L}{\partial w} = 2(\hat{y} - y)x = ?$
- $\frac{\partial L}{\partial b} = 2(\hat{y} - y) = ?$
- $w_{\text{new}} = w - \eta \frac{\partial L}{\partial w} = ?$
- $b_{\text{new}} = b - \eta \frac{\partial L}{\partial b} = ?$
- $\hat{y}_{\text{new}} = ?$
- $L_{\text{new}} = ?$

Observe: $L_{\text{new}} < L_{\text{old}} \rightarrow$ Good learning step! 😊

Big Picture

Input $\rightarrow$ Prediction $\rightarrow$ Loss $\rightarrow$ Gradients $\rightarrow$ Update parameters $\rightarrow$ Repeat

This cycle is the heart of learning in every neural network.

In the next lesson, we will learn how to compute gradients for all parameters in a multi-layer network using the

Chain Rule $\rightarrow$ Backpropagation.

